Music Wall

Application web collaborative pour
organiser et partager des découvertes musicales

Janna Gudumac – CDA RNCP – 2026

Project overview



- **WHO** – People who like discovering, organising and sharing music.
- **WHAT** – A full-stack web application to create shared musical walls.
- **WHERE** – Online, accessible from a browser on desktop and mobile.
- **HOW** – Angular · Spring Boot · PostgreSQL · Docker · JWT Security
- **WHY** –To organise music more visually and collaboratively than a simple playlist.

Une playlist ne suffit pas toujours

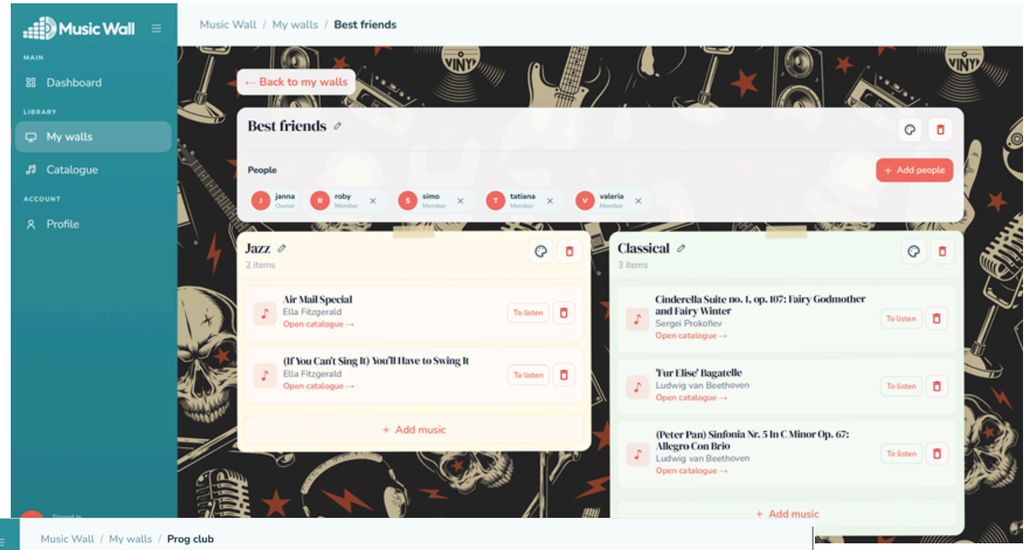
Les découvertes musicales sont souvent dispersées entre playlists, messages, recommandations et notes.

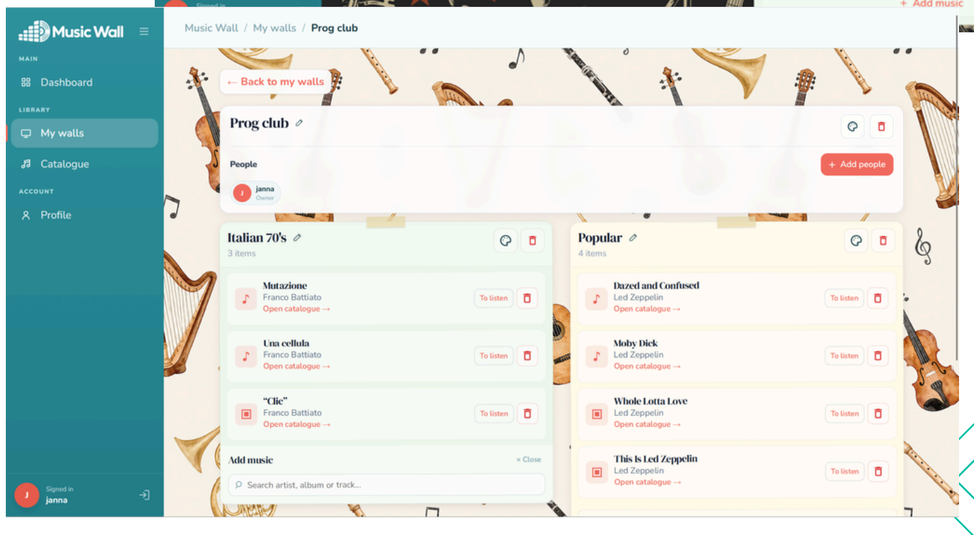
Problèmes identifiés

- Difficile d'organiser des recommandations par thème ou contexte
- Peu de possibilités de structurer une sélection musicale
- Besoin de partager un espace avec plusieurs utilisateurs
- Besoin de distinguer les contenus à écouter de ceux déjà écoutés

Ma solution – Music Wall

- Des murs musicaux partagés
- Des sections personnalisables
- Des albums et morceaux issus d'un catalogue
- Des statuts "To listen" et "Listened"
- Un accès contrôlé selon les droits de l'utilisateur





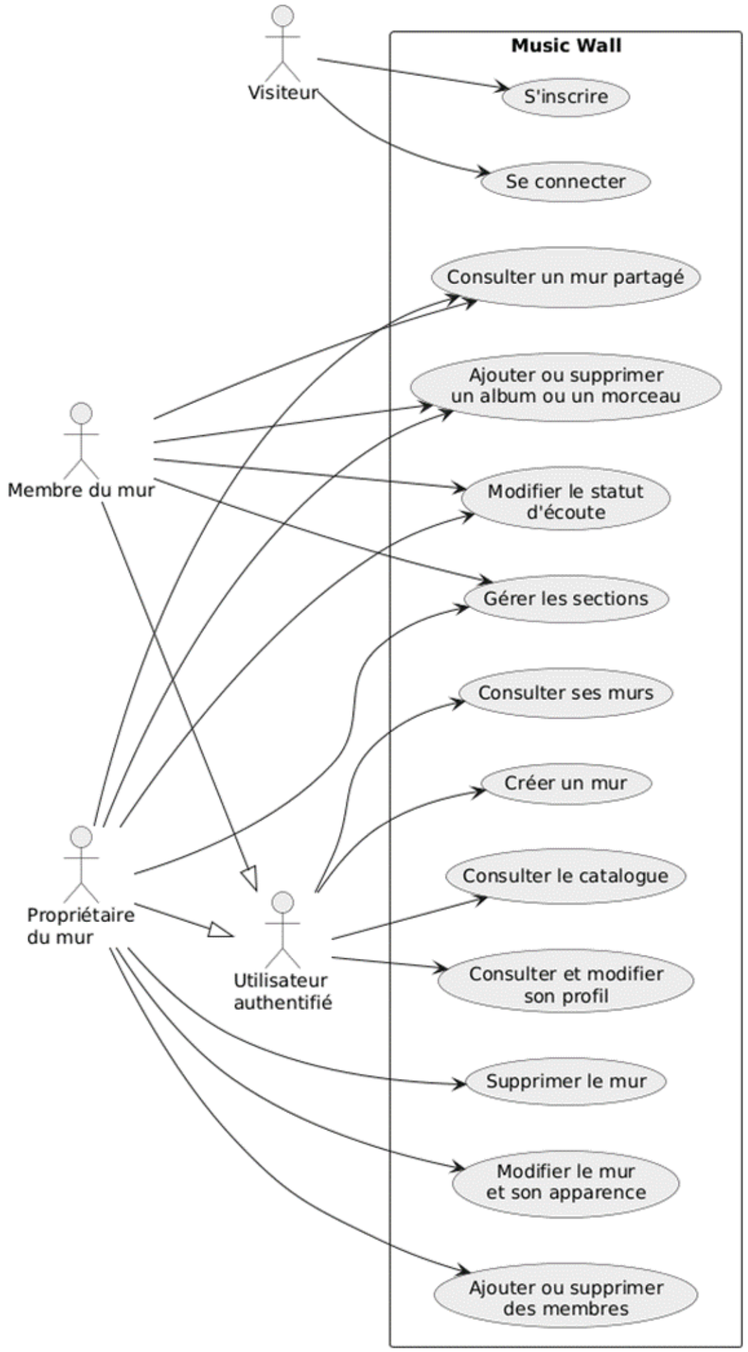
MVP / Version future

Fonctionnalités retenues

- Inscription et connexion
- Authentification JWT
- Profil utilisateur
- CRUD des murs musicaux
- Personnalisation des murs
- Gestion des membres
- Gestion des sections
- Ajout d'albums et de morceaux
- Statuts d'écoute
- Catalogue musical local
- Recherche
- Interface responsive
- PostgreSQL
- Docker Compose

Fonctionnalités reportées

- Système d'amis
- Invitations avec acceptation/refus
- Favoris
- Statistiques musicales
- Rôles plus avancés
- API musicale externe en temps réel



Cas d'utilisation

Visiteur

S'inscrire

Se connecter

Utilisateur authentifié

Consulter le catalogue

Gérer son profil

Créer un mur

Consulter ses murs

Propriétaire

Modifier le mur

Personnaliser son apparence

Ajouter ou retirer des membres

Supprimer le mur

Membre

Accéder au mur partagé

Gérer les sections et les contenus musicaux

Méthodologie

Le projet ayant été réalisé individuellement, je n'ai pas appliqué Scrum au sens strict. J'ai travaillé de manière itérative et incrémentale, en découpant le projet en fonctionnalités, en les priorisant, puis en les développant et en les testant progressivement. Git m'a permis de suivre l'évolution du projet et de conserver la traçabilité des modifications.

Étape 1

Authentification et profil utilisateur

Étape 2

Murs musicaux, membres et sections

Étape 3

Catalogue et éléments musicaux

Étape 4

Responsive, tests, Docker et déploiement

Outils

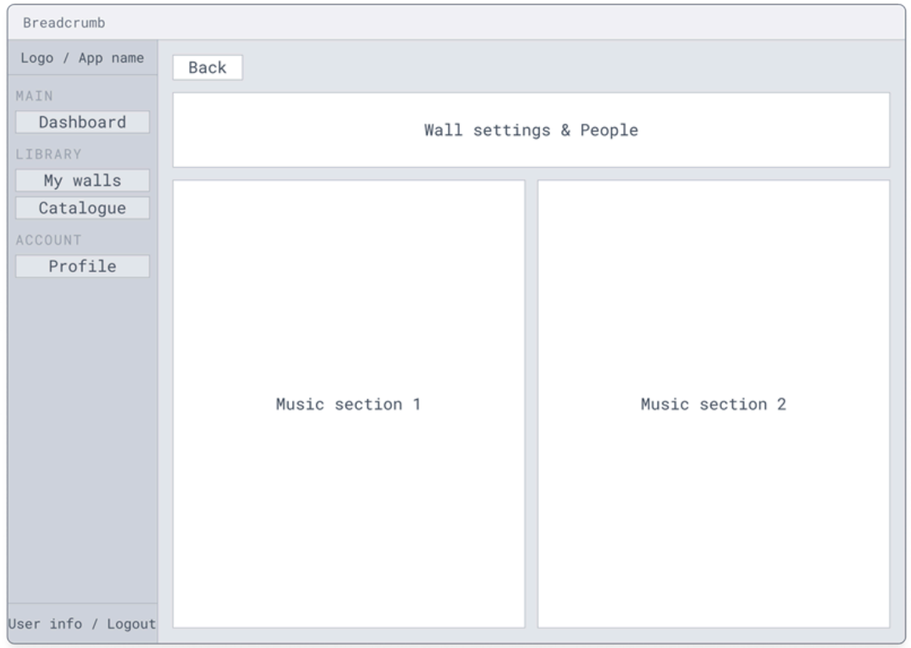
Git – GitHub – VS Code – Postman – pgAdmin – Docker

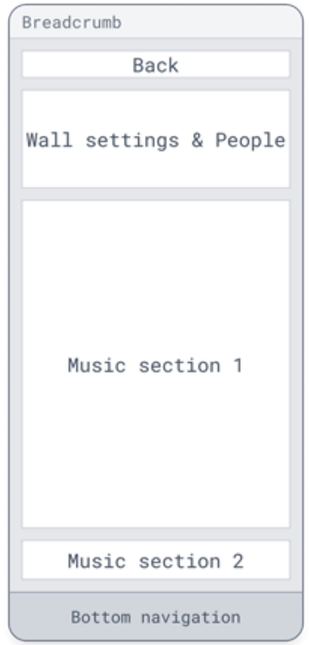
Conception graphique

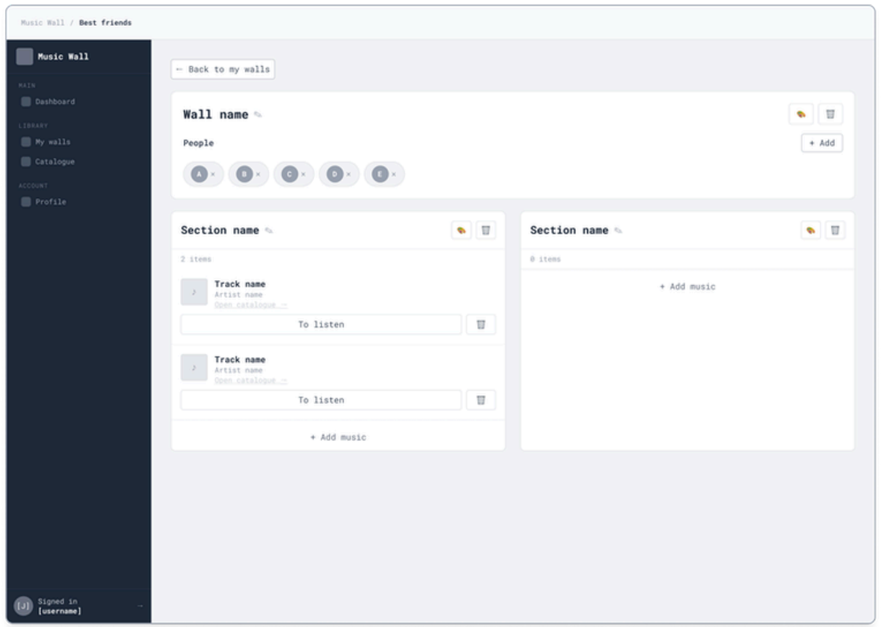
J'ai utilisé Figma pour préparer l'interface avant et pendant le développement.
J'y ai réalisé :

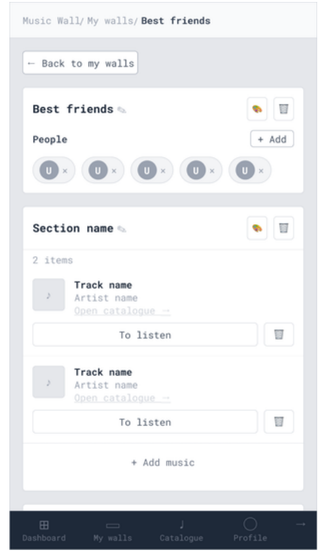
- le zoning
- les wireframes
- les maquettes
- les versions desktop et mobile
- la palette de couleurs
- la hiérarchie visuelle

Les maquettes ont ensuite servi de référence pour l'intégration en Angular et CSS.









Charte graphique

CORAL
Primary
#F36C5F

DARK PETROL BLUE
Text & headings
#173B44

VERY PALE BLUE
Main background
#DFF4F6

TURQUOISE
Sidebar / navigation
#258995

WHITE
Cards & surfaces
#FFFFFF

BLUE-GREY
Secondary text
#7DA6AF

FUNCTIONAL COLOURS — ERROR

Background	Border	Text
#FFF4F1	#F5B7AE	#A63E32

MUSIC SECTION COLOURS

Cream	Pink	Peach	Mint	Sky Blue	Lavender
#FFFBEA	#FFF0F3	#FFF2DF	#EEF9EC	#EEF7FF	#F4EFFF

Music Wall

logo

DM Serif Display

titres

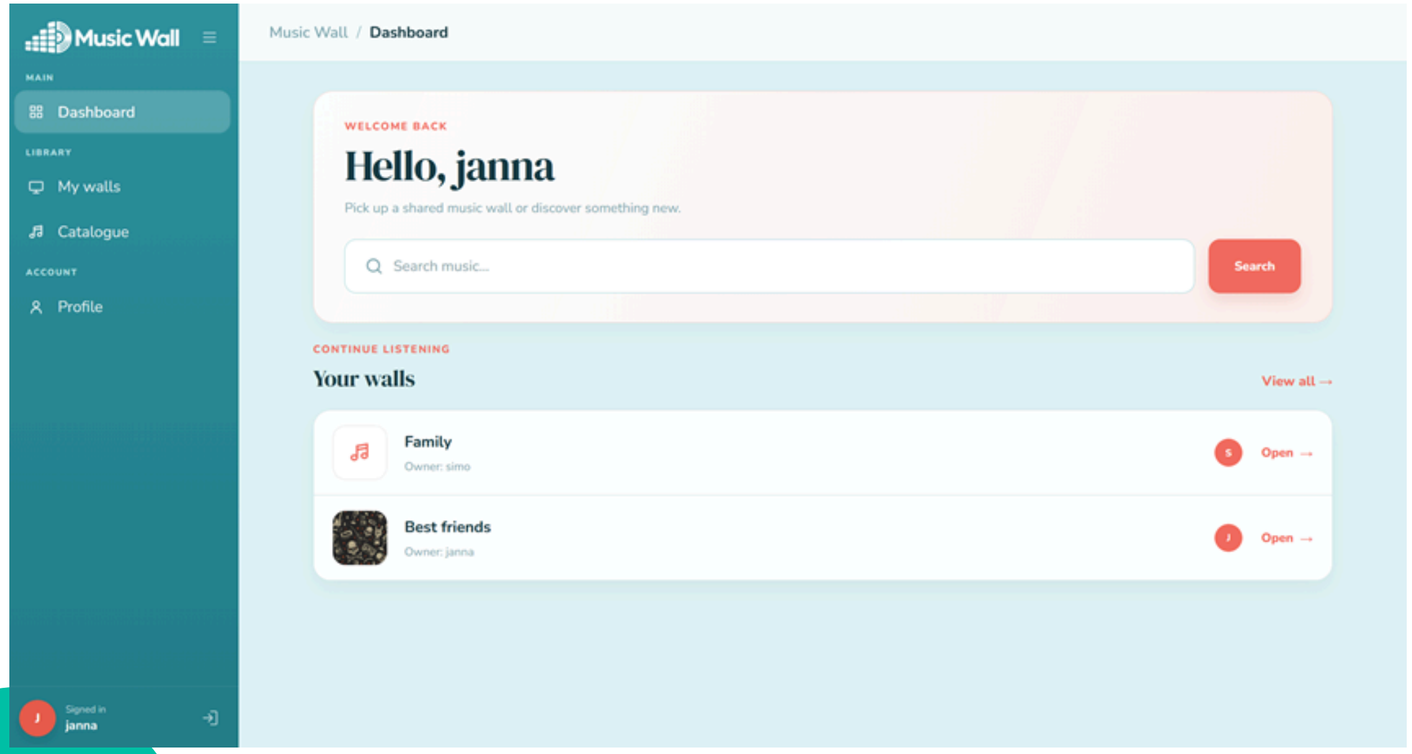
Nunito

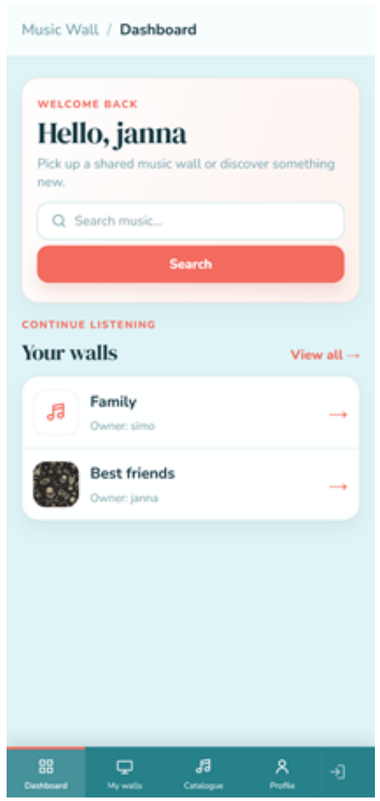
textes, boutons et navigation

Responsive design

Le concept de mur musical étant plus lisible sur un écran large, j'ai d'abord conçu la version desktop.
J'ai ensuite adapté l'interface aux smartphones avec :

- navigation fixe en bas
- réorganisation des boutons
- adaptation des cartes
- retour à la ligne des titres longs
- réduction des espaces
- adaptation des filtres et formulaires





Exemple: 2 vs 1 colonne:

```
.section-list {
  display: grid;
  grid-template-columns: repeat(2,
minmax(0, 1fr));
}

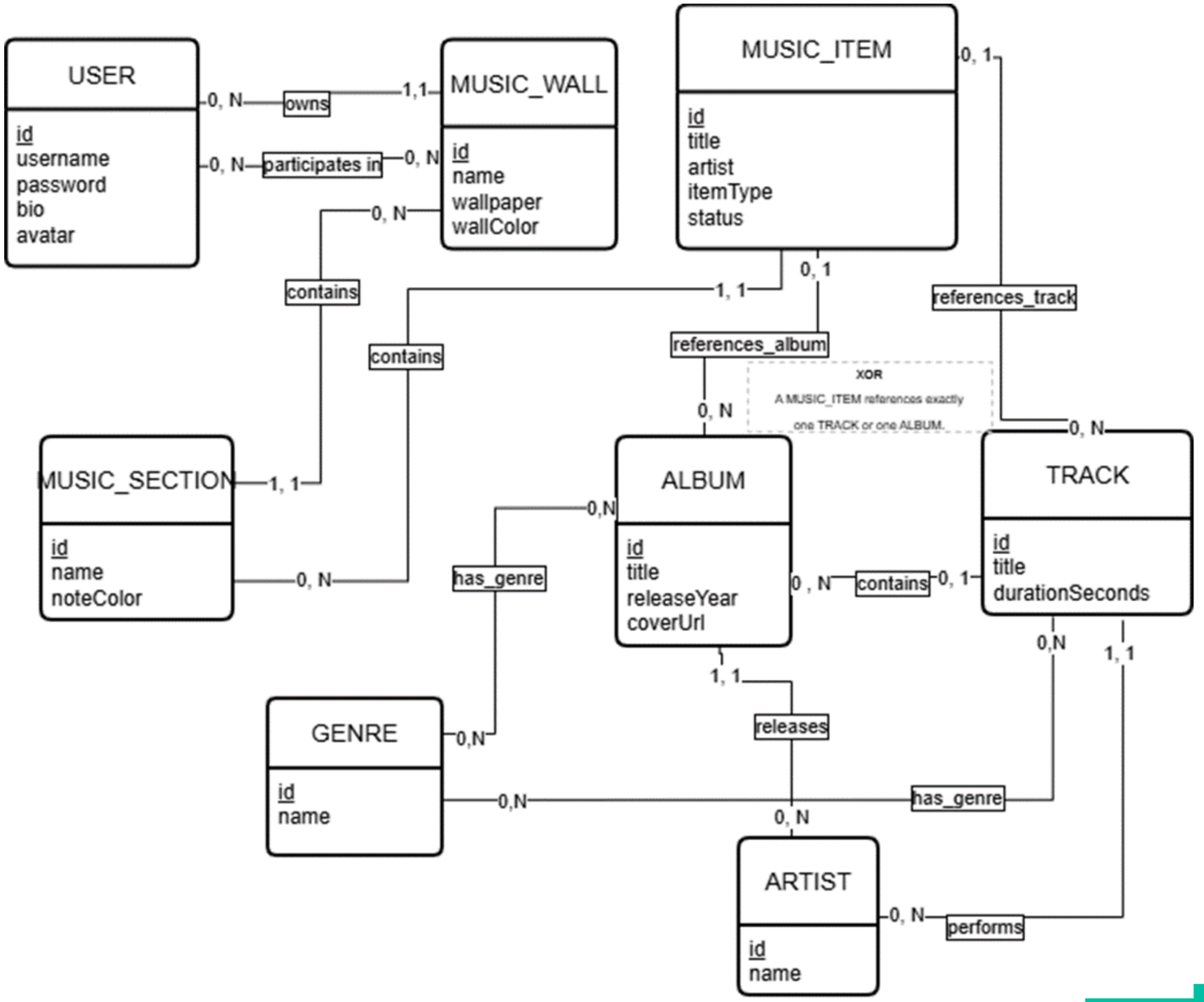
@media (max-width: 700px) {
  .section-list {
    grid-template-columns: 1fr;
  }
}
```

MCD – Modèle Conceptuel de Données

Les principales entités sont :
USER – MUSIC_WALL – MUSIC_SECTION – MUSIC_ITEM – ARTIST – ALBUM – TRACK – GENRE

Principales relations :

- un utilisateur peut posséder plusieurs murs
- un mur contient plusieurs sections
- une section contient plusieurs éléments musicaux
- un artiste possède plusieurs albums et morceaux
- albums et morceaux peuvent avoir plusieurs genres
- un MUSIC_ITEM référence soit un TRACK, soit un ALBUM



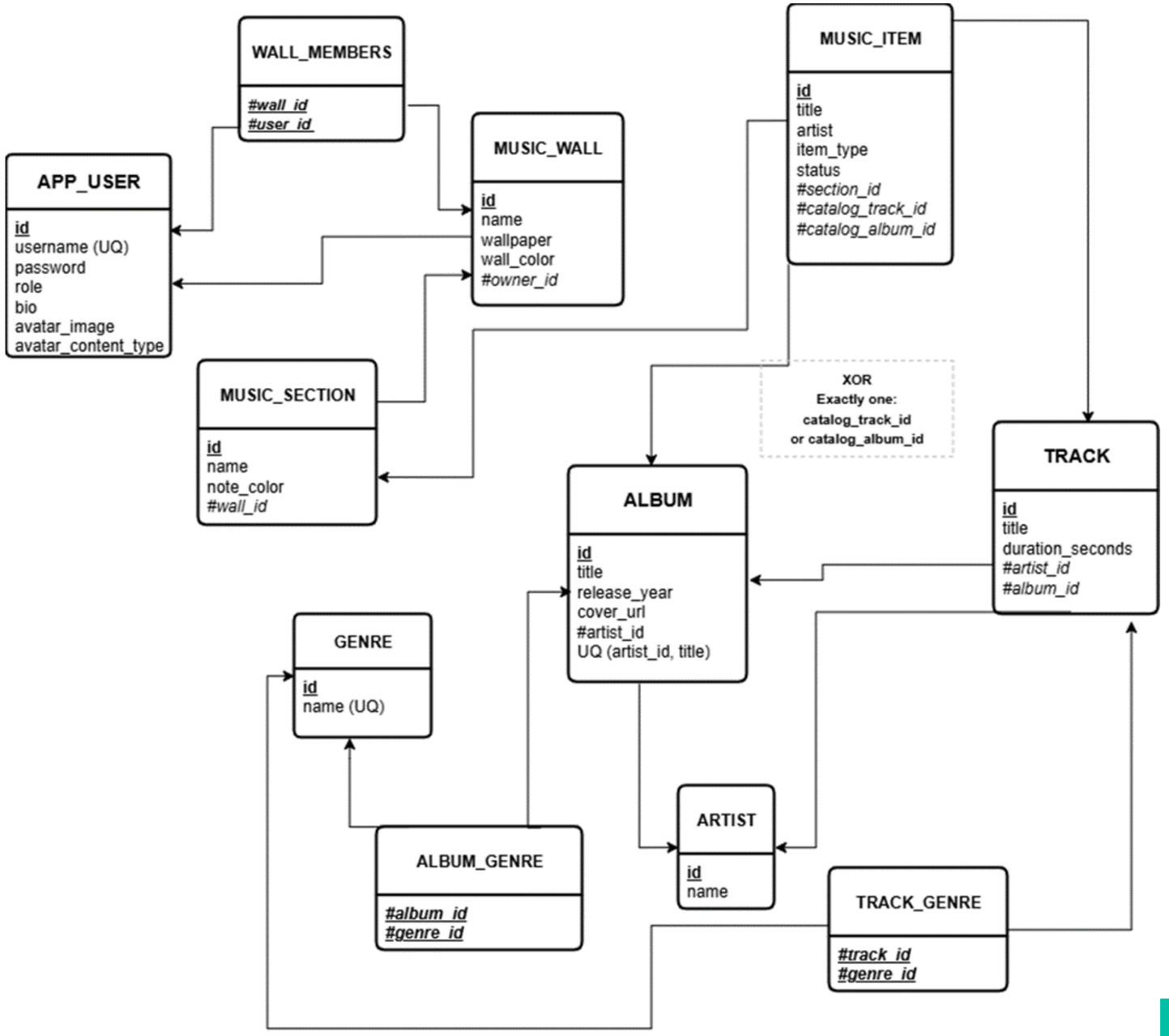
Du MCD au modèle relationnel

Les relations plusieurs-à-plusieurs nécessitent des tables d'association :

- WALL_MEMBERS
- ALBUM_GENRE
- TRACK_GENRE

Les relations un-à-plusieurs sont représentées avec des clés étrangères :

- owner_id
- wall_id
- section_id
- artist_id
- album_id



MPD – Modèle Physique de Données

La base physique est implémentée avec PostgreSQL.



Diagramme de classes

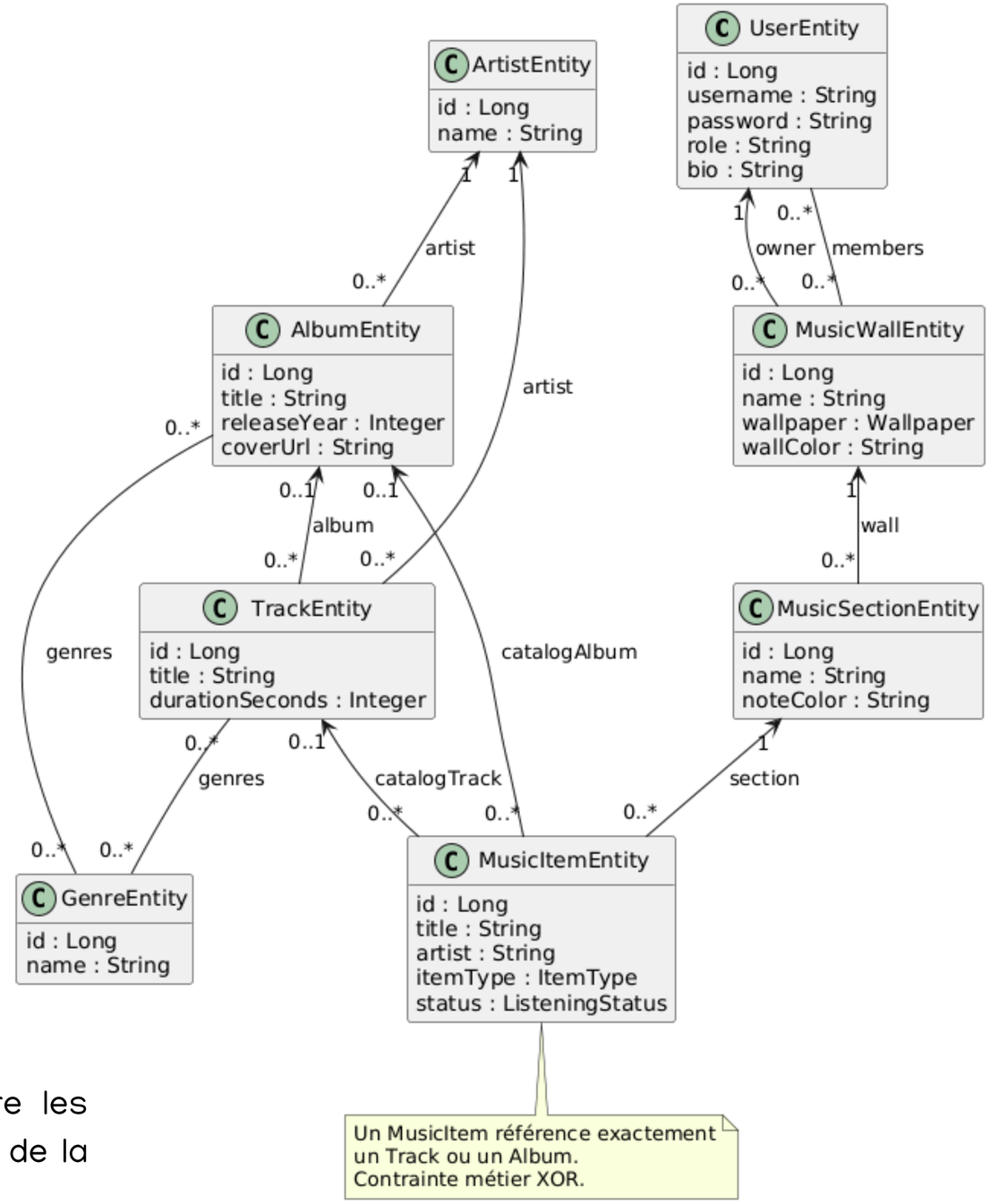
Le diagramme de classes représente la structure objet principale de l'application et les relations entre les entités Java.

UserEntity représente les utilisateurs.
MusicWallEntity représente les murs et leurs propriétaires ou membres.
MusicSectionEntity organise le contenu d'un mur.
MusicItemEntity représente un album ou un morceau ajouté à une section.
Les classes **ArtistEntity**, **AlbumEntity**, **TrackEntity** et **GenreEntity** représentent le catalogue musical.

```
@ManyToOne
@JoinColumn(name = "owner_id")
private UserEntity owner;
```

```
@ManyToMany
@JoinTable(name = "wall_members", ...)
private Set<UserEntity> members;
```

JPA traduit les relations entre les objets Java vers les relations de la base PostgreSQL.



Environnement technique

Frontend

Angular – TypeScript – HTML – CSS

Router – HttpClient – Forms – Guards – Interceptor

Backend

Java – Spring Boot – Spring Security – Spring Data JPA – Maven

Base de données

PostgreSQL

Déploiement

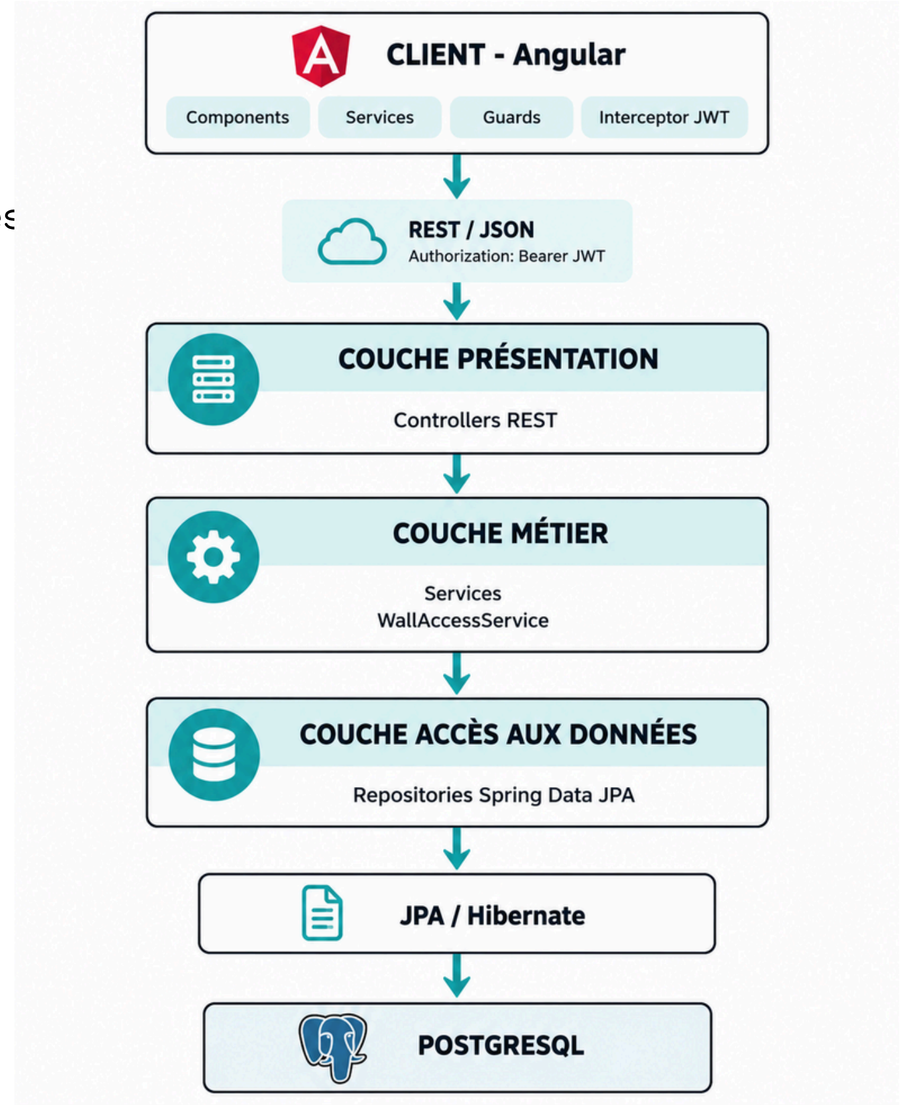
Docker – Docker Compose – Nginx – Netlify – Render

Outils

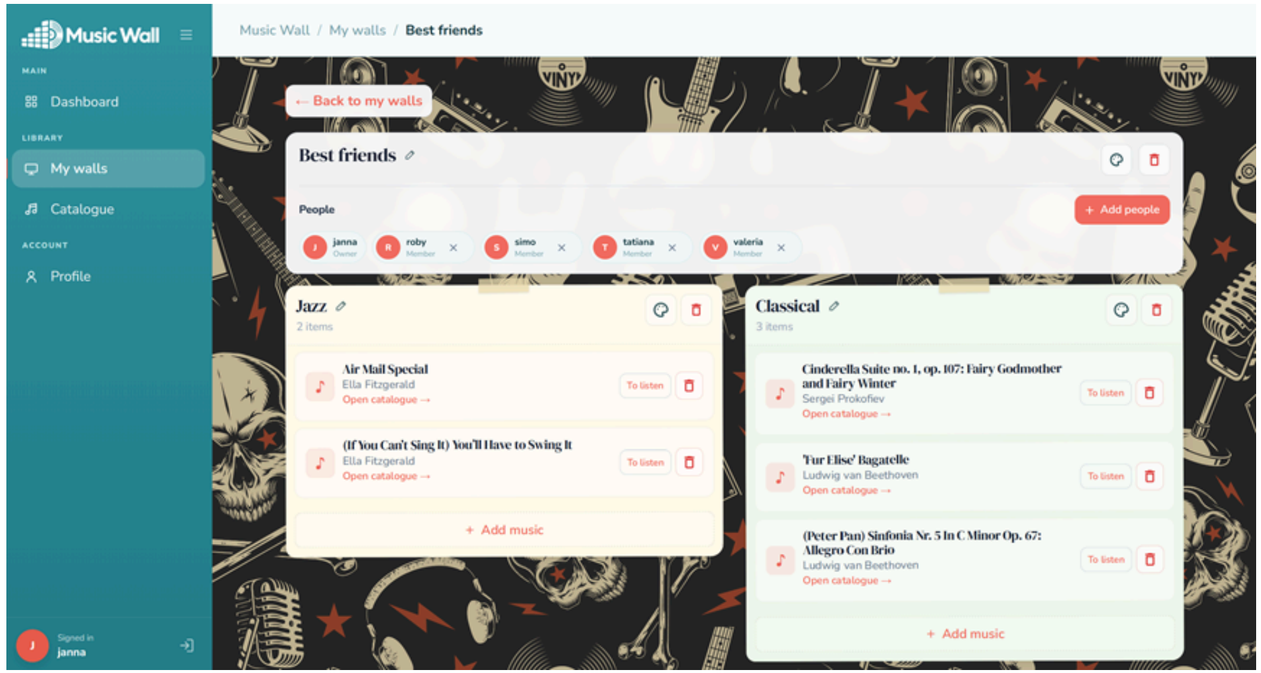
Git – GitHub – VS Code – Postman – pgAdmin – Figma

Architecture globale

```
music-wall/
├── backend/
│   ├── src/main/java/com/musicwall/
│   │   ├── controller/    points d'entrée HTTP
│   │   ├── service/       règles métier
│   │   ├── repository/    accès aux données avec Spring Data
│   │   │   └── projection/ résultats partiels de requêtes utilisés par les repositories
│   │   ├── entity/        modèle de persistance JPA
│   │   ├── dto/           contrats de requête et de réponse
│   │   ├── security/      JWT et Spring Security
│   │   └── exception/     gestion centralisée des erreurs HTTP
│   ├── src/test/          tests unitaires et tests d'intégration ciblés
│   └── Dockerfile
├── frontend/
│   ├── src/app/
│   │   ├── components/    pages et composants visuels
│   │   ├── services/      appels API et logique partagée
│   │   ├── models/        structures TypeScript utilisées côté frontend
│   │   ├── guards/        protection des routes
│   │   └── interceptors/  ajout du JWT aux requêtes protégées
│   ├── Dockerfile         build Angular + exécution avec Nginx
│   └── nginx.conf         fallback SPA et proxy des requêtes /api
├── database/              données d'initialisation du catalogue
└── docker-compose.yml     configuration et lancement des services Docker
```



Démonstration de Music Wall



- Connexion
- Création d'un mur
- Personnalisation du mur
- Ajout d'un membre
- Création d'une section
- Ajout d'un morceau depuis le catalogue
- Modification du statut d'écoute
- Affichage mobile

Architecture backend

Séparation des responsabilités

Controller

Reçoit les requêtes HTTP et retourne les réponses.

Service

Contient la logique métier.

Repository

Accède aux données avec Spring Data JPA.

Entity

Représente les données persistées.

DTO

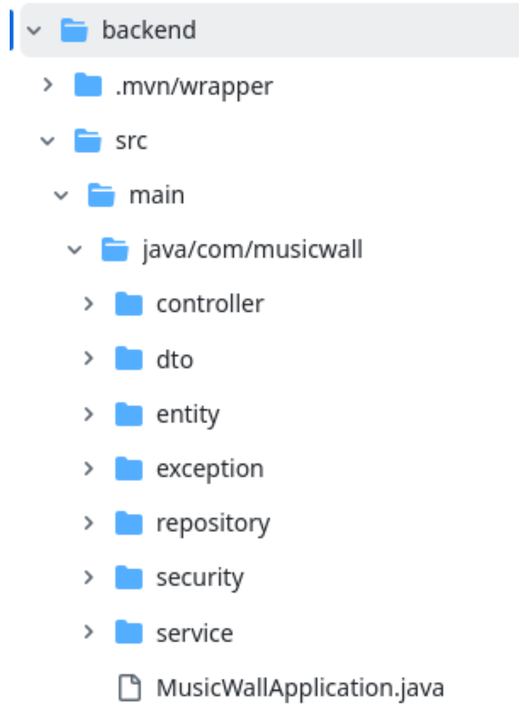
Définit les données échangées avec l'API.

Exception

Gère les erreurs métier et les réponses HTTP associées.

Security

Gère l'authentification JWT et Spring Security.



Code backend - Controller REST

```java
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public MusicWallDTO createWall(
        Authentication authentication,
        @Valid @RequestBody MusicWallDTO request
) {
    return musicWallService.createWall(authentication.getName(), request);
}
```

Le controller reçoit la requête HTTP et récupère les données envoyées par le frontend. Il ne contient pas la logique métier principale. Il délègue le traitement au service puis retourne le résultat au client.

Cette séparation permet de garder les controllers simples et de centraliser la logique métier dans les services.

Autorisation / Authentification



L'authentification vérifie qui est l'utilisateur. L'autorisation vérifie ensuite ce qu'il a le droit de faire.

Dans Music Wall, les droits dépendent principalement de la relation avec le mur : propriétaire, membre ou utilisateur extérieur.

Authentification

AuthenticationManager vérifie les identifiants. Si l'authentification réussit, on récupère l'utilisateur puis on génère la réponse contenant le JWT.

```
public AuthResponse login(LoginRequest request) {
    authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                    request.getUsername(),
                    request.getPassword()
            )
    );

    UserEntity user = userRepository.findByUsername(request.getUsername())
            .orElseThrow(() -> new BusinessException(message: "User not found"));

    return createResponse(user);
}
```

Figure 19 - Exemple : authentification - AuthService.java

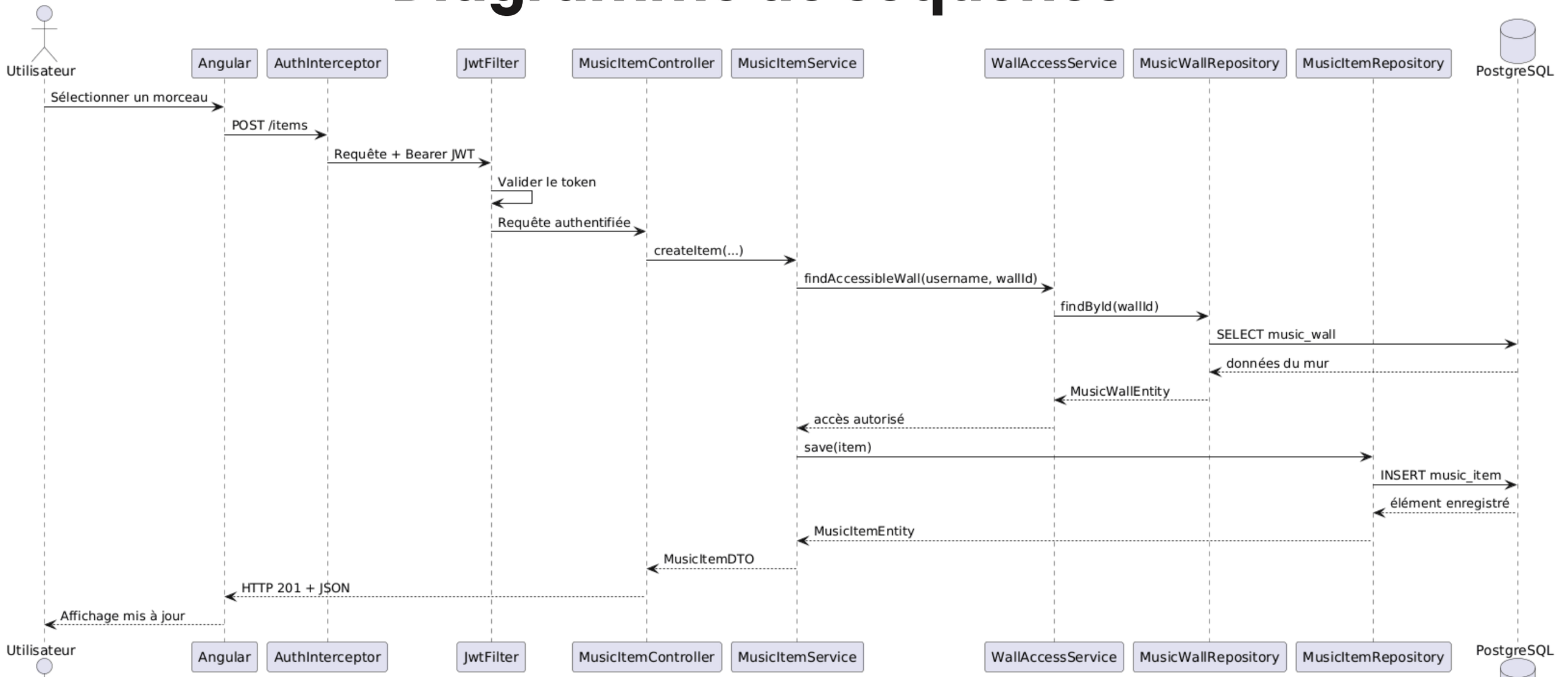
Autorisation

Le fait d'être connecté ne donne pas automatiquement tous les droits.
WallAccessService centralise les contrôles d'accès:

```java
public MusicWallEntity findOwnedWall(String username, Long wallId) {
    MusicWallEntity wall = findWall(wallId);
    if (!isOwner(wall, username)) {
        throw new ForbiddenException(
            message: "Only the wall owner can perform this action");
    }
    return wall;
}
```

Figure 20 - Exemple : autorisation métier - WallAccessService.java

Diagramme de séquence



Ajout d'un morceau à une section

L'utilisateur sélectionne un morceau dans Angular.
L'intercepteur ajoute le JWT à la requête.
Le JwtFilter valide le token côté backend.
Le Controller transmet la requête au Service.
WallAccessService vérifie que l'utilisateur peut accéder au mur.
Le MusicItem est enregistré dans PostgreSQL.
Le backend retourne le résultat au frontend.

Sécurité de l'application



Mots de passe

Les mots de passe sont hachés avec BCrypt avant stockage. Ils ne sont jamais enregistrés en clair.



JWT

Après connexion, un JWT est généré. L'intercepteur Angular l'ajoute aux requêtes protégées et le backend le valide.



Validation

Les données sont vérifiées côté frontend et côté backend. La validation backend reste indispensable.



Injection SQL

L'accès aux données passe par Spring Data JPA et Hibernate, sans concaténation directe des entrées utilisateur.



XSS

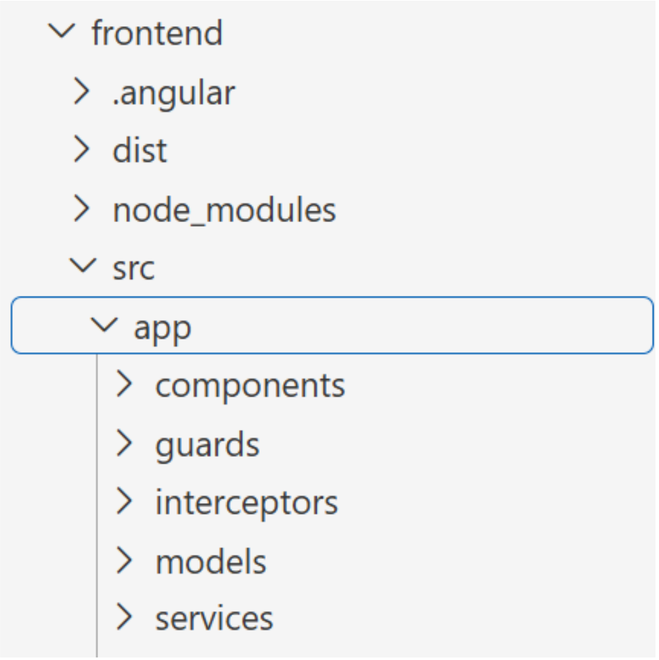
Angular échappe par défaut les valeurs affichées dans les templates, ce qui réduit les risques de scripts injectés.



CSRF & CORS

API stateless avec JWT dans le header Authorization, et backend limité à l'origine frontend configurée.

Frontend Angular



Components
Contient les composants visuels et les pages de l'application.

Guards
Protège certaines routes et empêche l'accès aux pages privées si l'utilisateur n'est pas authentifié.

Interceptors
Intercepte les requêtes HTTP, notamment pour ajouter automatiquement le JWT dans le header Authorization.

Models
Définit les structures TypeScript utilisées pour représenter les données échangées dans le frontend.

Services
Centralise la logique réutilisable et les appels HTTP vers l'API Spring Boot.

Recherche réactive avec Angular et RxJS

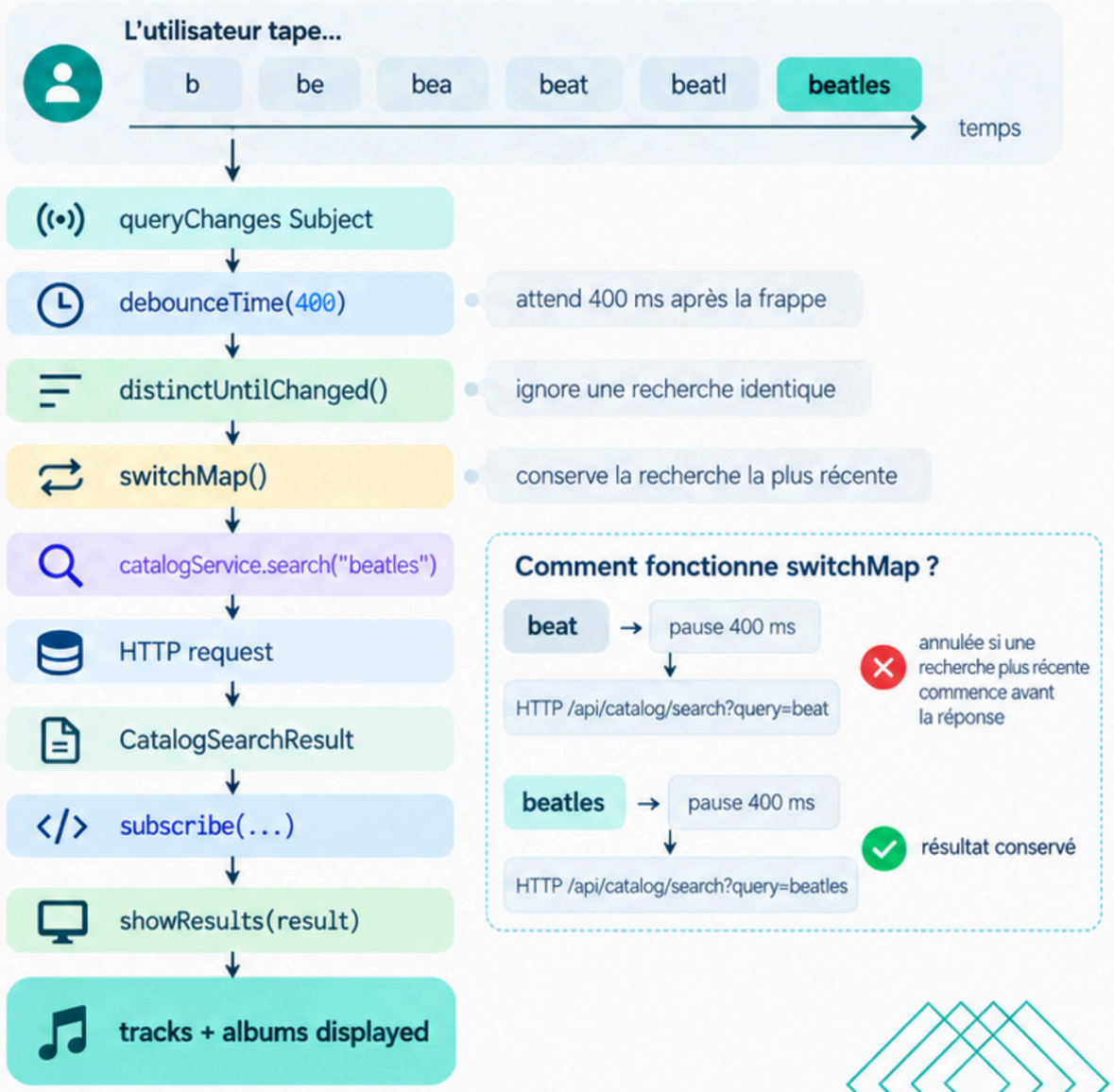
Une recherche rapide, efficace et réactive grâce aux opérateurs RxJS

catalog-search.component.ts

```
this.queryChanges.pipe(
   debounceTime(400),
   distinctUntilChanged(),
   switchMap(query =>
      this.catalogService.search(query)
   ),
   takeUntil(this.destroy$)
)
.subscribe(result =>
   this.showResults(result)
);
```

Points clés

1. `debounceTime(400)` attend 400 ms après la frappe
2. `distinctUntilChanged()` ignore une recherche identique à la précédente
3. `switchMap()` conserve la recherche la plus récente et abandonne l'ancienne
4. `takeUntil()` arrête l'abonnement lorsque le composant est détruit.



Logique métier – ajout d'un contenu musical

```java
private void applyRequest(MusicItemEntity item, MusicItemDTO request) {
    boolean hasTrack = request.getCatalogTrackId() != null;
    boolean hasAlbum = request.getCatalogAlbumId() != null;
    if (hasTrack == hasAlbum) {
        throw new BusinessException(message: "Choose exactly one track or album from the catalogue");
    }
```

Un élément ajouté à une section doit correspondre exactement à un morceau ou à un album du catalogue.

Cas valides : TRACK renseigné + ALBUM vide ou ALBUM renseigné + TRACK vide

Cas invalides : TRACK + ALBUM ni TRACK ni ALBUM

Cette règle est vérifiée dans **MusicItemService** avant l'enregistrement afin de conserver la cohérence des données.

Accès aux données avec Spring Data JPA

Dans Music Wall, je n'ai pas créé de classes DAO séparées. Les repositories Spring Data JPA assurent ce rôle. Ils étendent JpaRepository et permettent notamment :

- de lire les données ;
- d'enregistrer une entité ;
- de mettre à jour les données ;
- de supprimer une entité.

Hibernate réalise ensuite la correspondance entre les entités Java et les tables PostgreSQL.

MusicWallRepository permet de rechercher les murs appartenant à un utilisateur ou auxquels il participe. Le nom de la méthode décrit directement la requête : je récupère les murs dont l'utilisateur est propriétaire ou membre, sans doublons, triés du plus récent au plus ancien.

```java
public interface MusicWallRepository extends JpaRepository<MusicWallEntity, Long> {

    List<MusicWallEntity> findDistinctByOwnerUsernameOrMembersUsernameOrderByIdDesc(
            String ownerUsername,
            String memberUsername
    );
}
```

```
SELECT DISTINCT ...
FROM music_wall ...
WHERE owner.username = ?
  OR member.username = ?
ORDER BY id DESC;
```

Recherche en temps réel avec pg_trgm

Pour la recherche du catalogue, j'utilise une requête native PostgreSQL.

LIKE permet de trouver les correspondances partielles, tandis que l'opérateur % et la fonction similarity() de l'extension pg_trgm permettent de prendre en compte des noms proches et de classer les résultats par pertinence.

Les valeurs 3, 2 et 1 sont seulement des poids de tri. Comme le résultat est trié en ordre décroissant, la correspondance exacte reçoit le poids le plus élevé.

```
// Use PostgreSQL pg_trgm to find similar artist names and sort matches by relevance.
@Query(value = """
        SELECT a.*
        FROM artist a
        WHERE lower(a.name) LIKE '%' || lower(:query) || '%'
           OR lower(a.name) % lower(:query)
        ORDER BY
            CASE
                WHEN lower(a.name) = lower(:query) THEN 3
                WHEN lower(a.name) LIKE lower(:query) || '%' THEN 2
                WHEN lower(a.name) LIKE '%' || lower(:query) || '%' THEN 1
                ELSE 0
            END DESC,
            similarity(lower(a.name), lower(:query)) DESC,
            a.name ASC
        """, nativeQuery = true)
List<ArtistEntity> searchSimilar(@Param("query") String query);
```

Stratégie de tests

Tests unitaires

Les services métier sont testés de manière isolée avec JUnit et Mockito.

Les dépendances, notamment les repositories, sont remplacées par des mocks.

Exemples :

- gestion des murs musicaux ;
- ajout d'un morceau ou d'un album ;
- gestion du profil ;
- règles métier.

Tests d'intégration

Les tests Spring Boot + MockMvc vérifient plusieurs composants ensemble, depuis la requête HTTP jusqu'au traitement backend.

Exemples :

- inscription d'un utilisateur ;
- vérification du hash BCrypt ;
- changement de mot de passe ;
- validation des données envoyées à l'API.

Objectif

Détecter les régressions et vérifier que les différentes parties de l'application fonctionnent correctement ensemble.

```
[INFO] Results:
[INFO]
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  8.303 s
[INFO] Finished at: 2026-09-06T23:18:56+02:00
[INFO] ------------------------------------------------------------------------
```

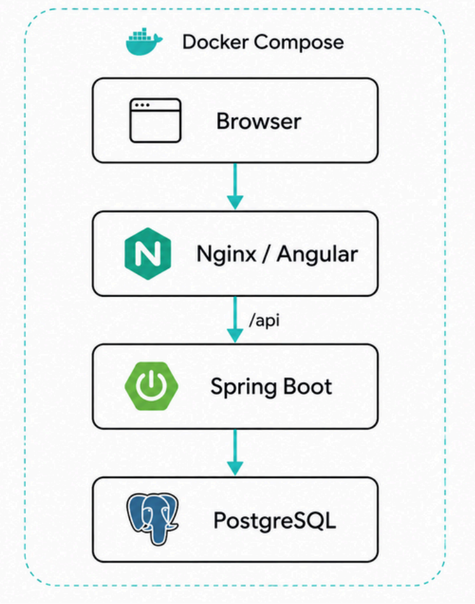
Docker et environnement local

```yaml
services:
  postgres:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USERNAME} -d music_wall_rncp"]

  backend:
    build:
      context: ./backend
    depends_on:
      postgres:
        condition: service_healthy

  catalogue-seed:
    image: postgres:16-alpine
    depends_on:
      backend:
        condition: service_healthy

  frontend:
    build:
      context: ./frontend
    depends_on:
      catalogue-seed:
        condition: service_completed_successfully
```



Docker Compose permet de lancer localement les différentes parties de Music Wall dans un environnement cohérent.

Frontend

Angular est compilé puis servi par Nginx.

Backend

Spring Boot expose l'API REST.

Base de données

PostgreSQL assure la persistance.

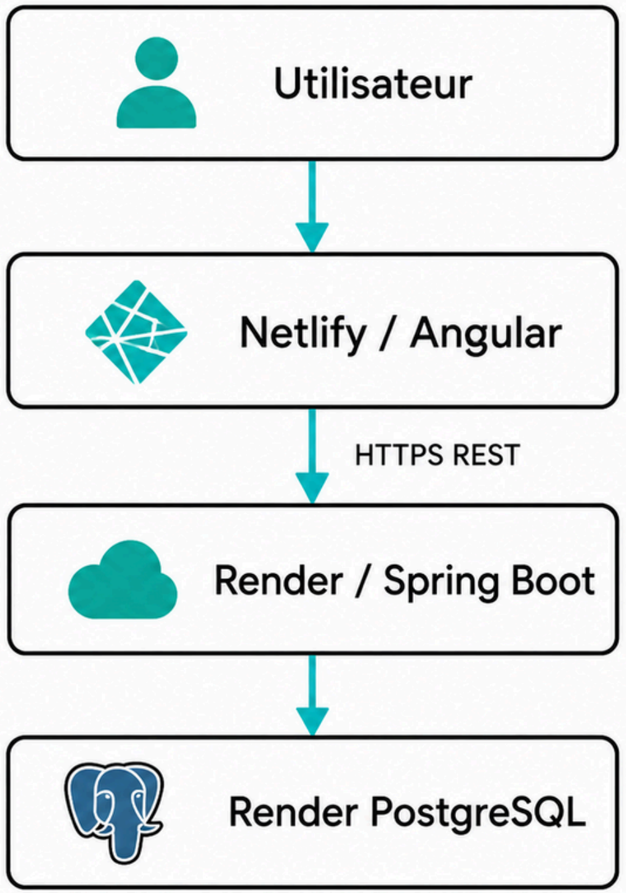
Catalogue

Les données du catalogue sont initialisées dans la base locale.

Nginx

Nginx sert l'application Angular et transmet les requêtes /api au backend.

Déploiement en production



Frontend – Netlify

Le build Angular est déployé sur Netlify.
https://music-wall.netlify.app/

Backend – Render

L'API Spring Boot est déployée sur Render.
https://music-wall-backend.onrender.com

Base de données – Render PostgreSQL

La base de production est indépendante de la base locale.

Variables d'environnement

Les informations sensibles et les paramètres dépendant de l'environnement sont configurés hors du code source.

CORS.

Le backend autorise l'origine du frontend Netlify.

Veille et recherche technique



Pendant le développement, j'ai utilisé principalement :

- la documentation officielle Angular ;
- la documentation Spring ;
- la documentation PostgreSQL ;
- la documentation Docker ;
- Stack Overflow et Reddit pour certains problèmes ciblés.

En complément, je suis plusieurs chaînes YouTube en différentes langues qui publient régulièrement du contenu sur les frameworks, les outils et les technologies modernes du développement web.

Exemple concret

Lors de la mise en place de la sécurité, j'ai recherché le fonctionnement de l'authentification JWT avec Spring Security et Angular : génération du token, filtre côté backend, interceptor côté frontend et gestion des routes protégées.

Difficultés rencontrées

1 – Définir un MVP réaliste

Le périmètre initial était trop large.

→ J'ai reporté les amis, invitations, favoris et statistiques pour conserver un projet cohérent.

2 – Responsive design

La structure d'un mur fonctionne naturellement mieux sur un grand écran.

→ J'ai réorganisé la navigation, les cartes et les actions pour les smartphones.

3 – Contrôle des accès

Être authentifié ne suffit pas pour avoir accès à tous les murs.

→ J'ai centralisé les contrôles propriétaire/membre dans WallAccessService.

4 – Catalogue musical

Je ne voulais pas que le fonctionnement principal dépende d'une API externe en temps réel.

→ J'ai utilisé un catalogue stocké localement dans PostgreSQL.

Conclusion

Music Wall m'a permis de réaliser une application full-stack complète, depuis la conception jusqu'au déploiement.

Le résultat :

- Une application full-stack fonctionnelle et déployée
- Une gestion sécurisée des murs partagés
- Une interface responsive desktop/mobile

Ce projet m'a surtout permis de comprendre comment les différentes couches d'une application communiquent entre elles.

Merci pour votre attention !

JWT – validation d'une requête

- Le JwtFilter intercepte les requêtes avant leur arrivée dans les controllers.
- Il extrait le JWT du header Authorization et vérifie sa validité.
- Si le token est valide, l'utilisateur authentifié est placé dans le SecurityContext.
- Le controller peut ensuite récupérer l'identité de l'utilisateur

```
String header = request.getHeader("Authorization");

if (header != null && header.startsWith(prefix: "Bearer ")) {
    String token = header.substring(beginIndex: 7);

    try {
        String username = jwtUtil.extractUsername(token);

        if (SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails =
                    userDetailsService.loadUserByUsername(username);

            if (jwtUtil.validateToken(token, userDetails.getUsername())) {
                UsernamePasswordAuthenticationToken authentication =
                        new UsernamePasswordAuthenticationToken(
                                userDetails,
                                null,
                                userDetails.getAuthorities()
                        );

                authentication.setDetails(
                        new WebAuthenticationDetailsSource().buildDetails(request)
                );

                SecurityContextHolder.getContext()
                        .setAuthentication(authentication);
            }
        }
            } catch (Exception exception) {
                // An invalid token simply leaves the request unauthenticated.
            }
        }

        filterChain.doFilter(request, response);
    }
}
```